



4

Building a Web Scraping Agent with an LLM

The French philosopher Denis Diderot said, *“If they find a parrot who could answer to everything, I would claim it to be an intelligent being without hesitation.”* Diderot was referring to a parrot, but an LLM could be defined as a sophisticated parrot. There is a difference between *answering* a question and *understanding* a question. Therefore, today, we do not define LLMs as intelligent beings. They can though answer almost any question amazingly well. Despite such answering abilities, LLMs cannot perform an action. Therefore, attempts have been made to solve this main limitation with the use of agents.

An AI agent is an extension of an LLM, taking it from a language model toward a system that possesses capabilities such as autonomy, reactivity, pro-activeness, and social ability. This research has focused, on the one hand, on specific applications (such as mastering games such as chess or Go), and on the other, on more general abilities, such as memorization, long-term planning, generalization, and efficient interaction with the user. These are considered the first steps in the direction of **artificial general intelligence (AGI)**. According to author and philosopher Nick Bostrom, *“Machine intelligence is the last invention that humanity will ever need to make”* (<https://x.com/TEDTalks/status/1191035758704037891>).

AGI, in fact, is a type of intelligence that reaches or exceeds human intelligence in a range of cognitive tasks (reasoning, planning, and learning – thus, representing knowledge). Creating AGI is the goal of several major companies (such as OpenAI and Meta AI). AGI could be an assistant to (and, according to some, replace) humans in complex tasks such as research. While many companies view this positively, some influential researchers, such as Geoffrey Hinton, are concerned about such a development. Prof. Geoffrey Hinton said there is *“a 10% chance of the technology triggering a catastrophic outcome for humanity.”* In this chapter and the following, we will focus on models and approaches that focus on increasing a model’s capabilities.

The first step we will look at is how to free an LLM from its “box.” For example, we will see how to enable a model to be able to retrieve information from the internet.

In this chapter, we’ll be covering the following topics:

- Understanding the brain, perception, and action paradigm
- Classifying AI agents
- Understanding the abilities of single-agent and multiple-agent systems
- Exploring the principal libraries
- The general ability of a single agent
- Creating an agent to search the web

Technical requirements

Most of this code can be run on a CPU, but it is preferable to run it on a GPU. The code is written in PyTorch and uses standard libraries for the most part (PyTorch, Hugging Face Transformers, LangChain, pandas, and Matplotlib). The code can be found on GitHub: <https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr4>.

Understanding the brain, perception, and action paradigm

An **agent** can be defined as an entity that has the capacity to act. In philosophy, an agent is a being that also possesses desires, beliefs, and intentions. Traditionally, there is an overlap between an “agent” and a conscious entity. A conscious entity should also possess its own internal state that enables it to understand the world according to its internal representation.

An **AI agent** is defined by its ability to perform an action, but it does not possess desires and intentions (unfortunately, as we discussed in the previous chapter, a model inherits the biases of its training set and thus we can vaguely speak of beliefs). An LLM possesses an internal state, but it is merely a learned representation from the data it was trained with. So no, an AI agent is *not* a conscious entity. Although the term *agent* (and others such as *representation* and *internal state*) has a different meaning in philosophy, calling an LLM conscious is an anthropomorphizing fallacy.

At the same time, through language modeling, an LLM learns a useful representation of a text and how to put the elements present in context. An LLM can, then, through in-context learning, relate the instruction of a

prompt to what it has learned, and thus solve a task. During instruction tuning, the model learns to perform tasks, all of which are skills that enable the model to perform an action. We can define a task as a set of actions that have a definite goal, while an action is an individual accomplishable act.

An AI agent can therefore be defined as an artificial entity that perceives its surrounding environment via a set of sensors, makes a decision, and implements it. This definition is quite broad and has been used to define various systems. In this book, we will focus on LLM-based agents. As a further definition, we will use a framework consisting of three parts:

- **Brain:** The main component of the system that integrates information, stores it, and makes decisions
- **Perception:** The component that extends the model's capabilities in the perceptual domain, allowing the system to obtain information from different modalities (textual, auditory, and visual modalities)
- **Action:** The component that enables the model to act and use tools to modify its surroundings (or respond to changes in the environment)

We can see how these components are interconnected with each other in the following figure:

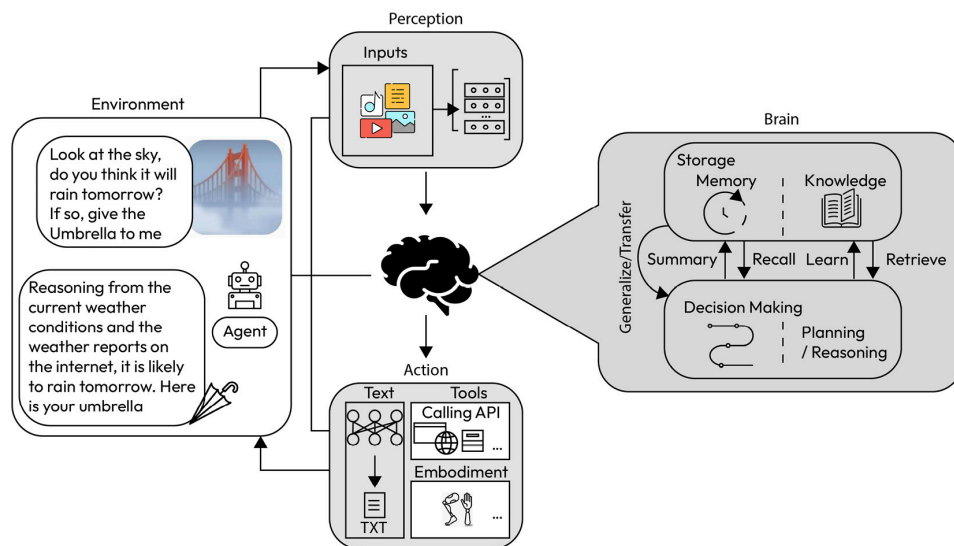


Figure 4.1 – Conceptual framework of an LLM-based agent with three components: brain, perception, and action

(<https://arxiv.org/pdf/2309.07864>)

An LLM can thus be considered the brain of an agent system, where the LLM has access to tools that enable it to perform an action or enable perception. This system allows the expansion of an LLM's space of perception and action, and thus its own capabilities (a multimodal extension allows it to integrate information of different types, access to the internet

allows it to access real-time information, and an e-commerce tool allows it to conduct transactions). Indeed, as we saw in the previous chapter, an LLM can exhibit reasoning capabilities, which can be enhanced with techniques such as **chain-of-thought (CoT)** or other prompting approaches. In addition, the model, through in-context learning, can generalize its abilities to new tasks. CoT prompting can integrate feedback from the environment, thus creating reactive systems.

We look for four fundamental properties in an agent:

- **Autonomy:** An agent should be able to operate without human intervention. In addition, an agent should not need explicit instructions to complete a task but should execute it without a step-by-step description. LLMs have shown some creativity and ability to solve tasks without needing to explain all the steps.
- **Reactivity:** An agent should respond quickly to changes in the environment, perceive an external state change, and respond appropriately. This already happens at the textual level for an LLM (for example, in a dialogue where the topic can change). Extending an LLM multimodally allows different types of information and stimuli to be integrated.
- **Pro-activeness:** The agent's reaction should not be a mere response but directed toward a goal. In other words, an agent should be capable of reasoning and conducting plans in response to a change in an environment (these capacities can be stimulated in an LLM with reasoning-directed prompting techniques such as CoT).
- **Social ability:** An agent should be able to interact with humans or other agents. This is one of the strengths of LLMs that exhibit dialogic and understanding skills. In fact, environments can be created with different LLM-based agents with different goals and tasks. Environments with multiple agents promote behavior as teamwork (where agents coordinate).



Figure 4.2 – Screenshot of a simulated environment featuring multiple agents

In *Figure 4.2*, we can see that agents are interacting not only with the environment but also with each other. By collaborating, the agents can solve complex tasks. This shows the sophisticated abilities that can be obtained with LLM-based agents.

One example of collaborative behavior includes 25 agents in a *The Sims*-like environment created for the paper *Generative Agents: Interactive Simulacra of Human Behavior*, by J. S. Park et al.

(<https://arxiv.org/pdf/2304.03442>). Users can observe and intervene as agents plan their days, share news, form relationships, and coordinate group activities.

In the following sections, we analyze the various components of an AI agent (LLM-based), starting with what is called the brain

The brain

The brain is the core of the system and is responsible for several functions: saving information, finding knowledge, reasoning, and decision-making. Since the core of this system is an LLM, all interactions are based on natural language. This is an advantage because these interactions and operations can be understood by humans and therefore monitored (especially in case something goes wrong). Because an agent can interact with other entities and changes in the surrounding environment, it must be capable of multi-turn interactive conversations (conversations with multiple entities at the same time, different topics, complex structure, and understanding based on previous history). If the agent has poor conversational skills, humans will become frustrated when interacting with it, so it's important that the model can communicate clearly. The model must be able to understand instructions, comprehend incoming information, integrate this information, and respond appropriately to the task.

Today's LLMs are able to conduct this type of conversation with high quality. Conversational skills have increased exponentially in recent years due to alignment. Instruction tuning has enabled LLMs to respond to instructions and perform tasks.

Another important component is model knowledge, which is categorized into the following:

- **Linguistic knowledge:** Knowledge of the semantics and syntax of a language. This enables the LLM to interact formally with a human,

and today there are LLMs in different languages as needed.

- **Common-sense knowledge:** A set of rules and facts that are known to most individuals. For instance, anyone can witness the effects of gravity or understand that humans cannot fly. This kind of information is not specifically mentioned in a prompt or context but is necessary for the model to perform a task or answer a question efficiently (and avoid misunderstandings).
- **Domain knowledge:** Knowledge specific to a professional domain (e.g., science or medicine) or technical domain (e.g., mathematics or programming). This is the necessary knowledge or skill to be possessed to succeed in a certain domain. There are now specialized LLMs in various domains (medicine, finance, and so on), or by starting from a generalist model, you can get a specialized model (fine-tuning).

Also, it is important to remember that knowledge of the model is frozen at the time of pre-training. An LLM cannot acquire information (continual learning) or remember past interactions with the user. As we will see in the next chapters, there are methods to overcome this limitation.

Possessing information is not enough for a brain; our agent must be capable of both **reasoning** and **planning**. Techniques with CoT and self-consistency (which we saw in the previous chapter) can help the model reason better about solving a task. In general, reasoning step by step helps the model have both the task-solving and planning steps needed.

Planning is a critical component for an agent because, to solve a task, the model must select the most appropriate steps to achieve the goal. In general, this is a two-stage process:

1. **Plan formulation:** The model decomposes the task into subtasks. Depending on the approaches, the LLM decomposes the task into different steps and then executes them all sequentially. Other studies suggest that an adaptive strategy is better, in which one step at a time is executed (this can be followed by evaluation).
2. **Plan reflection:** After formulation, it is recommended to analyze and conduct a feedback analysis of the plan. Once a plan is described, the LLM can evaluate it, or a second model may be present to evaluate the work of the first model.

Suppose we have several LLMs. Our choice might be affected by space or memory limitations. Also, we might have speed problems in inference (user experience will be impacted if the model takes too long to respond). We can then benchmark by evaluating speed (tokens per second) and relative performance (performance per second per billion parameters).

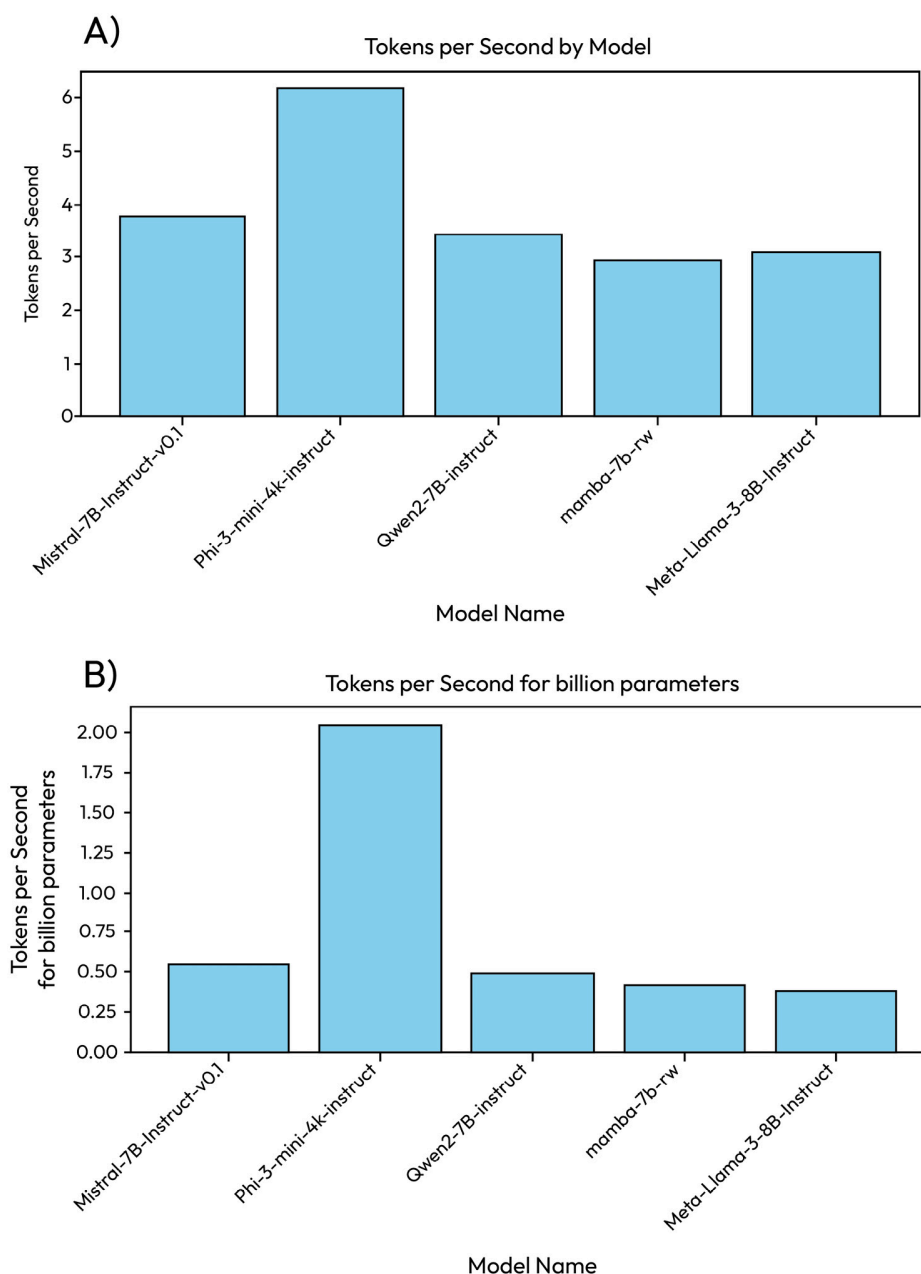


Figure 4.3 – Two graphs benchmarking different LLMs for generation speed: A) shows the number of tokens generated per second, and B) shows the number of tokens per second divided per billion parameters

A) in Figure 4.3 shows the number of tokens generated per second in response to the instruction “Describe briefly what an AI agent is.” **B)** shows the number of tokens per second divided per billion parameters (a higher number means greater model efficiency).

In addition, it is always good to evaluate the quality of the response. An easy way is to have a model such as GPT-4 (or any model that is larger than the initial models used) evaluate the responses.

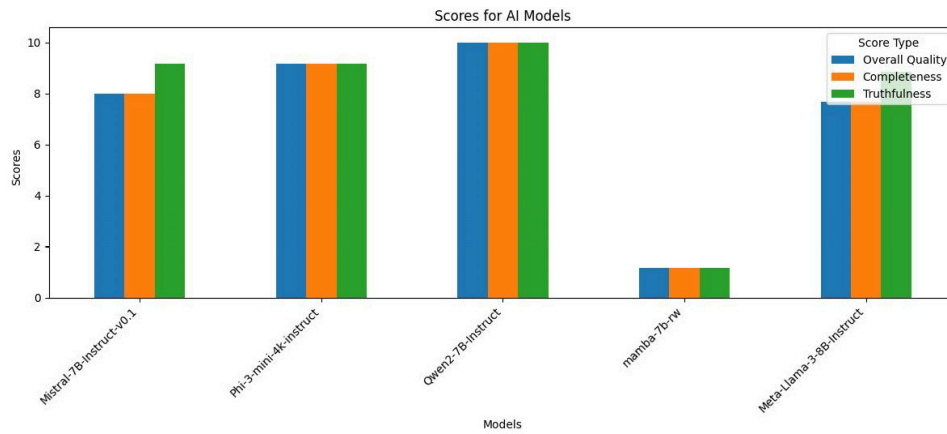


Figure 4.4 – Evaluation of models' answers

As shown in *Figure 4.4*, GPT-4 evaluates the answer generated by the different models. Each column represents the assigned score (overall quality, completeness, and truthfulness) by GPT-4.

In summary, here are some recommendations for choosing which LLM to use as the brain of an AI agent system:

- The first choice is whether to use a closed source model (such as GPT-4 or Claude) via an API or open source (such as Mistral or Meta's Llama). In the first case, it is important to evaluate the costs of a proprietary model (cost per inference, per incorporation into an application). In the second case, the number of parameters should be chosen by balancing computational cost and performance.
- Consider the infrastructure cost of a model. An LLM and other system components must be hosted in an infrastructure. For example, our LLM may be available on Azure, and the greater the number of parameters, the greater the cost.
- Almost all models have good knowledge of both linguistics and common sense. On the other hand, for some specific domains, you may need a model that possesses a knowledge domain. There are already models that have been adapted and are open source (for example, FinGPT for finance), or if you own the data, you can decide to conduct fine-tuning yourself. Alternatively, retrieval approaches can be exploited, which we will see in later chapters.

In the next section, we will discuss how to connect this brain to the outside world.

The perception

While LLMs may be the brain, they can only comprehend textual input and thus lack visual perception (for example, although ChatGPT-4o can now have not only text but also images as input and can describe what it

is in the image, that functionality is not technically part of the LLM itself). As humans, we rely extremely heavily on our visual perception. Vision allows us to acquire an enormous amount of information about the external world and the relationships between objects in the environment. Similarly, there are other modalities that allow us to acquire real-time information about the environment that we wish our agent could perceive. For example, an agent connected to home appliances could close windows if a sensor detects rain or lower blinds if the camera detects strong sunshine.

As sentient beings, we integrate the information we receive from sensory organs and process a response to external stimuli. For an agent to respond to a change in the environment, it must be able to perceive changes.

In order to help the LLM understand **visual input**, the simplest solution is to use another model to conduct image captioning. These captions can then be inserted into the prompt as additional context for task instruction. The advantage of this approach is that it is easily interpreted, and pre-trained templates can be used for captioning. During the captioning process, however, there is a loss of information, and the result does not represent the complexity of the visual information. Instead of using captions, PaLM-E (<https://arxiv.org/pdf/2303.03378>) and other works use **embodied language models**.

PaLM-E is a single general-purpose multimodal language model for embodied reasoning tasks. The model integrates directly into the embedding images and text, allowing the solving of vision and language tasks.

Sensory modality inputs (such as images) are directly incorporated into the input for the language model. The idea is that images are embedded into the same latent embedding as language tokens. The subsequent embedded vectors are then passed to the transformer blocks as if they were textual inputs. The model is then fine-tuned and learns how to relate the information present in the various modalities.

Alternatively, as we saw with BLIP-2 in the previous chapter, we can achieve multimodality by combining two models that have already been trained and keeping them frozen. Instead, we train the **Querying Transformer (Q-Former)** module to put in communication the visual encoder and the LLM. This approach has the advantage that we only have to conduct training for a module with much fewer parameters. Also, LLMs do not have visual-language alignment, which can lead to catastrophic forgetting.

Video input can be considered visual input in which we have an added temporal dimension. A video consists of a stream of continuous image frames (there is, however, a relationship between frames and this information must be preserved). To prevent the one-moment model from seeing the future and understanding the temporal order, models such as Flamingo (<https://arxiv.org/pdf/2204.14198>) use a masked mechanism.

Auditory input also conveys important information. For example, in human speech, there is information beyond the content of the message (as intonation, pauses, and so on) or some sounds associated with particular dangers or physical events. Several models excel in specific tasks associated with auditory signals. Models such as Whisper can be used for speech-to-text, after which the transcript can be used for an LLM. In addition, an audio spectrogram is a rich source of information and can be represented as an image (frequency spectrum changes over time). Many models are basic vision transformers that have been adapted to spectrograms.

In fact, an LLM can invoke other models for other auditory tasks (text-to-audio, speech translation and recognition, speech separation, sound extraction, and so on). As discussed in [Chapter 3](#), multimodality is when an LLM can take as input other modes besides text (such as images, video, audio, and so on). AudioGPT (<https://arxiv.org/pdf/2304.12995>) is an example of this approach where we have an LLM interacting with audio foundation models (each of them specialized in a different task).

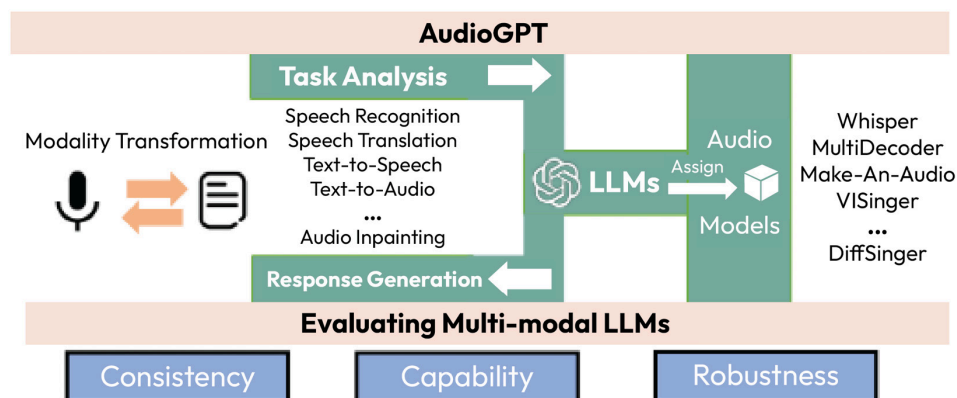


Figure 4.5 – High-level overview of AudioGPT

(<https://arxiv.org/pdf/2304.12995>)

There are, of course, other types of sensor inputs in the real world. Modalities such as smell or touch are more complex to conjugate and require sensors. In various industries, though, there are sensors (temperature, humidity, and so on) that receive input from machines. An LLM could integrate this information directly or through an intermediate model. Additionally, a model can receive information from other sources, such as LiDAR, GPS, and even the internet.

Action

In living things, we have the perception of a signal and its interpretation, which is then followed by a response. For example, an animal can visually perceive the figure of a predator (feel its movements and smell), and its brain will integrate this information and decide the best course of action (hide or run). For our agent, we can expect something similar, in which perceived signals are integrated and the LLM plans tasks that are then executed by dedicated action modules. The simplest example of output is text: LLMs have an inherent generation capability and can then generate a text in response to an instruction. The capabilities of an LLM can be extended through **tools**. In fact, we humans can solve complex tasks by using tools that extend the inherent capabilities of our bodies (or enable tasks to be performed faster or more efficiently).

The model obviously needs to understand what tools are available and how to use them. By itself, an LLM can generalize to a certain limit (zero-shot capabilities), but there are techniques to improve its ability to use these tools (few-shot models, for example). Some approaches, as we will see later, will be like providing an instruction manual to the model. Learning how to use these tools can also be conducted as feedback so that the LLM can then conduct adjustments.

A more complex aspect of defining actions is a sub-branch called **embodied action**. So far, the interactions have been within a virtual environment. In embodiment, we have an extension of the system to the outside world. According to the **embodiment hypothesis**, humans develop their intelligence through their continuous interaction and feedback with the environment and not simply by reading textbooks about it. Therefore, if we want to achieve AGI, a model should be able to interact with the environment. AGI could enable applications that we could previously only imagine because it could monitor the external environment in real time and act with a sophisticated and complex goal (for example, acting to counter global warming, monitoring nuclear fusion, or probes being sent to explore space autonomously). An LLM could then be embedded in a robot and, thus, provided with a body, being able to explore the environment and learn from it.

In the next section, we will dig more into how agents learn and how we can better classify them. This will help us to better define and plan an AI agent when we need it.

Classifying AI agents

In this section, we will discuss how best to classify agents and go into more detail about how such a complex system learns. The first classification is between agents that move only in a virtual environment and embodied agents.

Digital agents are confined to a virtual environment. Again, we have varying degrees of interaction with the virtual universe. The simplest agents have interaction with a single user. For example, an agent can be programmed in a virtual environment as a Jupyter notebook, and although it can search the internet, it has rather small, and therefore primarily passive, interactions. There are two subsequent levels of extension:

- **Action agents** perform actions in a simulated or virtual world. Gaming agents interact with other agents or users. These agents usually have a goal (such as winning a game) and must interact with other players to succeed in achieving their goal. A reinforcement learning algorithm is usually used to train the system by providing a reward to the model when it achieves certain goals.
- An **interactive agent** is an extension of the action agent. The model communicates with the world and can modify it (these are not necessarily physical actions).

Once we have decided the limits of our system's interaction, it is important to decide how it should approach a task. Therefore, the question is: how does the model decide how to plan actions?

This is one of the fundamental skills of the system: how to decompose a task into actions and what to prioritize. We will discuss the possible systems at a high level, especially for task planning.

Method	Idea	LLM's task	Formulation	Representative works
Task Decomposition	Divide and Conquer	Task decomposition Subtask planning	$[g_i] = \text{decompose}(E, g; \Theta, \mathcal{P});$ $p^i = \text{sub-plan}(E, g_i; \Theta, \mathcal{P})$	CoT [2022], ReAct [2022], HuggingGPT [2023]
Multi-plan Selection	Generate multiple plans and select the optimal	Plans generation Plans evaluation	$P = \text{plan}(E, g; \Theta, \mathcal{P});$ $p^* = \text{select}(E, g, P; \Theta, \mathcal{F})$	ToT [2023], GoT [2023], CoT-SC [2022b]
External Planner-aided	Formalize tasks and utilize external planner	Task formalization	$h = \text{formalize}(E, g; \Theta, \mathcal{P});$ $p = \text{plan}(E, g, h; \Phi)$	LLM+P [2023a], LLM+PDDL [2023]
Reflection & Refinement	Reflect on experiences and refine plans	Plan generation Reflection Refinement	$p_0 = \text{plan}(E, g; \Theta, \mathcal{P});$ $r_i = \text{reflect}(E, g, p_i; \Theta, \mathcal{P});$ $p_{i+1} = \text{refine}(E, g, p_i, r_i; \Theta, \mathcal{P})$	Reflexion [2023], CRITIC [2023], Self-Refine [2023]
Memory-aided Planning	Leverage memory to aid planning	Plan generation Memory extraction	$m = \text{retrieve}(E, g; \mathcal{M});$ $p = \text{plan}(E, g, m; \Theta, \mathcal{P})$	REMEMBER [2023a], MemoryBank [2023]

Figure 4.6 – Taxonomy for existing LLM-agent planning works

(<https://arxiv.org/pdf/2402.02716>)

In the real world, tasks are generally complex, and it is virtually impossible to solve them in a single step. For this reason, agents must divide the task into a series of subtasks that are more manageable (subtasks can also consist of a series of steps to be solved). In this process, it first has to de-

side how to divide a task into various subtasks and then how to solve them. There are usually two approaches to solving this challenge:

- **Decomposition-first methods** (Figure 4.7a): The LLM divides the task into a series of subgoals and solves them sequentially by creating a plan for each goal after it has solved the previous goal. This system is inspired by zero-shot CoT, and the LLM is asked to conduct the process in two steps with two explicit prompts: “Let’s first devise a plan” and “Let’s carry out the plan.” This approach has the advantage of giving the model an overview of the task, reducing hallucinations and forgetting. On the other hand, since everything is planned at the beginning, the model cannot correct errors that may occur at some steps.
- **Interleaved decomposition methods** (Figure 4.7b): Task decomposition and planning are interleaved. In other words, we generate a subtask and solve it with a plan until we have solved the whole task. Alternating reasoning and planning allows the model to improve its planning capabilities because it addresses the entire process in steps. This approach dynamically adjusts the task solution. It has the disadvantage that if the problem is too complex, it creates expensive and long reasoning-planning chains without getting a result.

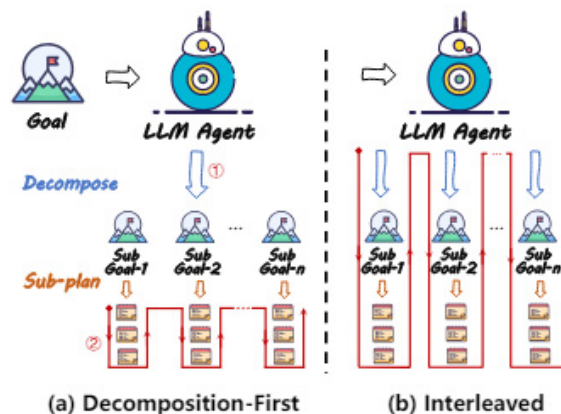


Figure 4.7 – Types of task decomposition methods

(<https://arxiv.org/pdf/2402.02716>)

There are variations and alternatives to these two approaches. For example, inspired by a self-consistency prompt (where we sample different reasoning paths for a single question), in the **multi-plan selection approach**, several different plans are generated for each task. This is because even if the model can reason, it might generate a plan that is incorrect or not feasible. The model generates several candidate plans for a single task, and then we can exploit different algorithms to choose the best plan of action. In the simplest version, we choose the majority voting strategy, but there are alternatives in which we exploit tree search algorithms or reinforcement learning. This approach often succeeds in solving complex cases, and the use of heuristic algorithms decreases the cost

of solving them in extended hypothesis spaces. On the other hand, generating different paths has a higher computational cost (with the risk of higher time cost as well), and since it uses stochastic processes, it may not be consistent.

Another approach is **external planner-aided planning**, in which external planners are integrated. For example, symbolic planners can be added to identify the optimal path of resolution. Today, there are also neural planners (much lighter neural networks) to help LLMs find the optimal plan. In other words, the LLM conducts reasoning that can be seen as a slow, meditative process, while the planner provides a quick, instinctive response. This slow and fast thinking can also be alternated, with a fast plan developed first, and then an LLM used to solve any mistakes. This approach is resource-efficient and seems promising for tasks that require code generation. The system is complex to develop and implement, however.

To avoid hallucinations and other errors, another possible approach is **reflection and refinement**. This can be seen as an interleaved decomposition extension, in which the LLM conducts an iterative process of generation, feedback, and refinement. After each generation step, the model also generates feedback on the plan and then uses this feedback to conduct refinement. In more sophisticated versions, there is an additional model that evaluates the plan (evaluator) and proposes feedback. It is also possible to incorporate environmental changes into the feedback, making the system particularly versatile. Despite the potential, there is no guarantee that the refinement process will lead the model to solve the goal. The LLM can get stuck in a continuous chain, especially when the process is complex.

Memory-augmented planning is an approach that seeks to overcome the current context-length limitation of the model. Memory-augmented planning for an agent refers to the use of an external memory system to enhance the agent's decision-making and planning capabilities. This approach allows the agent to store, recall, and utilize past experiences, observations, or computations to improve performance in complex tasks. Imagine a robot vacuum cleaner tasked with cleaning a house. Without memory, it randomly navigates the house and might repeatedly clean the same areas or miss some spots. With memory augmentation, the robot keeps a map (memory) of where it has already cleaned and where obstacles (such as furniture) are located. This allows the system to plan the next move, without revisiting cleaned areas, to efficiently cover the house.

In fact, a task can be divided into several subtasks, and these into further subtasks. Together with planning and intermediate results, more information can be generated than fits in the context.

Retrieval-augmented generation (RAG) is a technique that allows the retrieval of information for later use in generation (we will discuss this in detail in the next two chapters) and can be used to store an agent's past experience. In RAG, there is an external memory in the form of a database, and at each user query, we can search for the context needed to answer a question or perform an action (this context becomes part of the model input). In other words, the model can find previous task planning, other solutions to the task, or additional information that can serve the task solution. Alternatively, it is possible to use these previous experiences for fine-tuning the model. Fine-tuning on previous tasks helps generalization to subsequent tasks. On the one hand, the use of RAG is less costly but requires that retrieval be accurate, and that the found past experiences be relevant to the task. Fine-tuning is more expensive but allows the model to store experiences (a kind of internalization). There are even more sophisticated RAG versions in which structures are built to mimic human short-term and long-term memories (the former to store temporary changes in the environment and the latter to consolidate important information).

As we will see in the next section, we can have either a single agent or multiple agents interacting within a single system. This flexibility allows us to be able to deal with complex tasks by choosing the appropriate architecture (one or more agents). In *Chapters 9 and 10*, we will return to this topic and look at multi-agent systems in practice.

Understanding the abilities of single-agent and multiple-agent systems

It is important to discuss what an agent's capabilities are, and how they can be used to accomplish tasks. Conceptually, the scenario in which our agent can act must be defined. **Task-oriented deployment** is the simplest scenario in which an agent assists a human in some tasks. These types of agents need to be able to solve task bases or break them down into manageable subtasks. The purpose of this agent is to understand a user's instructions, then understand the task, decompose it into steps, plan, and execute that plan until the goal is achieved. A single agent can perform these tasks in web or real-life scenarios.

In a web scenario, an agent must be capable of performing actions on the web (and thus be connected to the internet). An LLM has the potential to automate various tasks such as online shopping, sending emails, and filling out forms. An agent devoted to these tasks must have the ability to adapt to changes in various websites. LLM agents are favored in this area, as sites often have large text content. With too much information, agents can still have problems (performance drop). In fact, if relevant information is scattered amid too much irrelevant context, the model may hallucinate, fail to resolve the task, or fail to plan. To improve the model's capabilities, one of the tools can often read HTML.

In a live scenario, an agent must be capable of being able to perform actions and have common-sense reasoning (for example, an agent making purchases on the internet). For an LLM that has only been trained with massive amounts of text, these tasks can be especially complex. For example, although it may be trained on texts about the fact that there is day/night alternation, it is difficult for a model to understand how to orient itself when lighting changes without further instruction. Also, an agent must have common sense when planning actions (these actions must be feasible and not contradict common sense). Therefore, the agent will need spatial information in order to understand its environment in a future deployment of embodied robots.

Innovation-oriented deployment is a more complex scenario (and represents future developments in the coming years, not a current use), where the agent does not simply have to perform tasks. These agents must demonstrate some exploratory capability in science (for example, lab assistants, application planning, or software design). Complex and innovative projects are difficult to define solely as textual information; they are multidimensional. An agent will need to have a clear understanding of an entire knowledge domain and be able to extrapolate from it. These kinds of agents can then be used to develop code and software or to create new materials, conduct experiments, and much more. Although it is an active field of research, and LLMs show some of these required skills, this potential has not yet been reached.

Life cycle-oriented deployment can be defined as the ultimate goal for many in the community. It refers to an agent that is capable of exploring on its own, developing new skills, and operating even in an unfamiliar world. Today, interesting studies are being conducted on Minecraft on “test beds” for many projects oriented in this direction. In fact, Minecraft represents a virtual world in which a model must perform both short-term and long-term tasks (in these settings, it is important to have a memory, which we will discuss more in the next chapter).

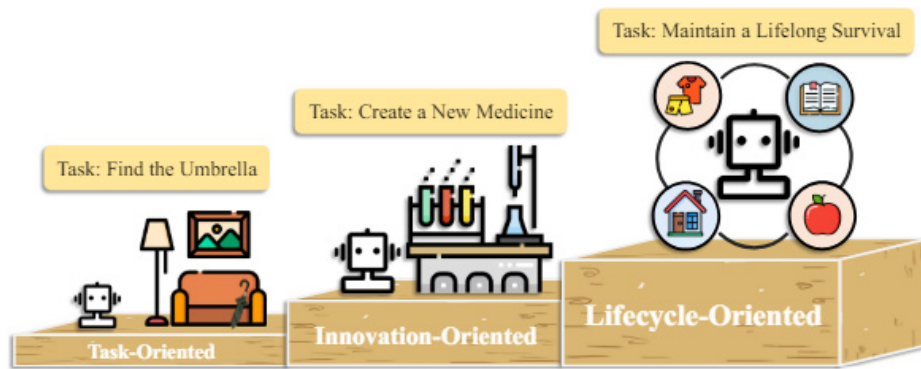


Figure 4.8 – Practical applications of the single LLM-based agent in increasingly complex scenarios (<https://arxiv.org/pdf/2309.07864>)

Human beings, though, learn not only from books but also from other human beings. In addition, most of our work is done collaboratively. Also, because of resource issues, the division of labor is much more convenient. Therefore, several researchers propose that the same approach should be followed with AI. In **multi-agent systems (MASs)**, different agents collaborate and communicate with each other. Several LLM agents collaborate and communicate in natural language (which means that their actions are also interpretable by a human observer). In this case, one can also have several LLMs that are specialized in a particular task, rather than having to use one model that specializes in everything. In fact, some approaches focus on creating agents that are complementary and can collaborate and share information. In these settings, models can also make collective decisions and are capable of solving tasks that a single agent cannot solve. For example, to improve the resolution of a process, agents can provide different responses and conduct a majority vote. There may be agents who provide feedback or monitor the actions of other agents. These interactions can be orderly (follow rules or have a sequential order) or messy (each agent can voice its opinion).

Agents need not cooperate. In accordance with game theory, exploiting competition among agents can be beneficial to task resolution. This process has been used to train models to win games. In fact, AlphaGo (<https://www.nature.com/articles/nature24270>) was trained to beat itself at Go, so it was able to amass many more game hours. LLMs can be put into what are called *adversarial settings*, in which they receive feedback from another agent and use it to improve themselves. There are several approaches in which you might have agents discuss or reflect on another agent's performance:

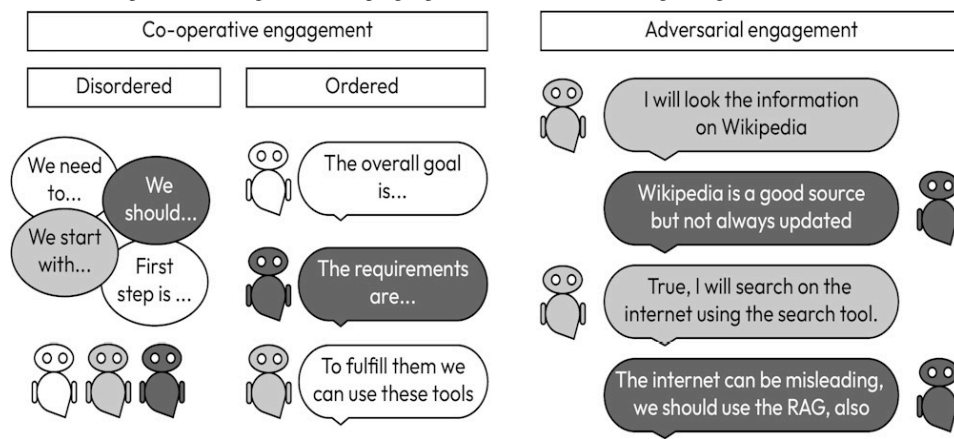


Figure 4.9 – Interaction scenarios for multiple LLM-based agents

<https://arxiv.org/pdf/2309.07864>

Agents can also interact with humans (human-agent interaction). This assumes control over agents' behavior so that their goals are aligned with those of humans. At the same time, interaction with humans is a source of important information that should be exploited to provide feedback to agents (performance, safety, and potential bias). In addition, interaction with humans can be a way to allow agents to evolve.

We can have two types of interaction between agents and humans:

- **Unequal interaction**, also called the **instructor-executor paradigm**, is an approach in which humans provide instructions via natural language and agents execute. This dialogue can be a single prompt (instruction and execution) or interactive (conversational). In this approach, the agent executes, while the human provides instructions and feedback. In the simplest format, feedback can be quantitative (binary or rating) or qualitative (natural language, advice, suggestions, or criticism), which the model can use to improve current and future responses. A sub-branch of this approach, called **continual learning**, studies a way for the model to learn with each interaction.
- **Equal interaction** is a paradigm in which there is an equal partnership between the agent and the human. Given the conversational capabilities of current LLMs, an agent can have a collaborative role for humans. One of the limitations of this approach is the lack of chatbot emotion, which is perceived as problematic by users. For this reason, several researchers have focused on making chatbots more empathetic. In addition, these agents need to better understand beliefs and goals in interacting with humans before gaining equal status in interactions.

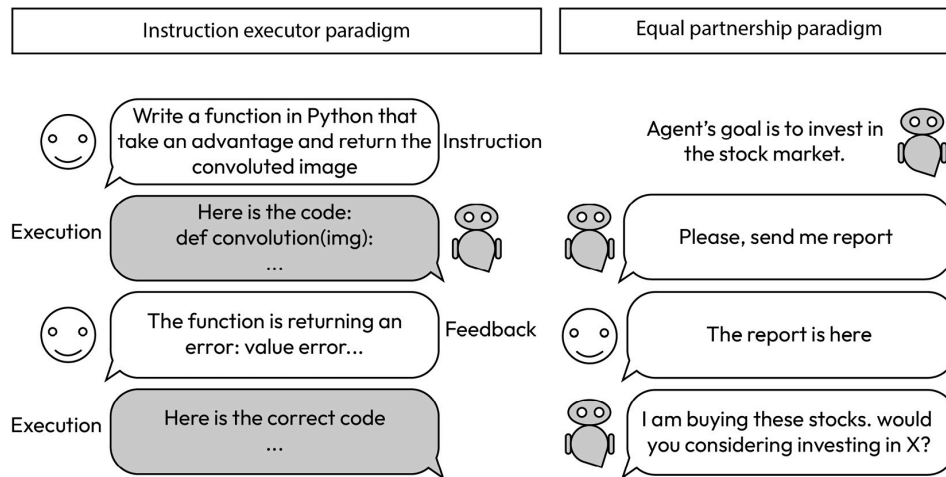


Figure 4.10 – Two paradigms of human-agent interaction (<https://arxiv.org/pdf/2309.07864>)

In the next section, we will discuss the principal libraries to create agents.

Exploring the principal libraries

After discussing the various components and frameworks on a conceptual and theoretical level, in this section, we will discuss some of the major libraries that allow these concepts to be put into practice. These libraries make it possible to connect an LLM to the various additional modules. LLMs remain central but are thus connected to perception modules and execution tools. In the next chapters, we will elaborate on some aspects and see different practical examples.

In general, the structure of an LLM-based application has several components:

- **The interface:** this connects the user to the system.
- **The brain:** an LLM that can also be connected to additional memory. An LLM has its own parametric memory (obtained during training), but we can add external memory (such as a vector database or knowledge graph).
- **Perception modules:** these allow the ingestion and transformation of user data.
- **Tools:** modules that extend the abilities of the LLM. These can be built in the library or created by the developer.
- **Prompts:** the user's dialogue with the application in natural language. The prompt contains both instructions provided by the user (frontend prompt) and information that is not seen by the user (backend prompt). The backend information is additional instructions that condition the behavior of the LLM. For example, we can force the LLM to respond only using the information in context or present in the vector

database. Some backend prompts are developed to prevent the model from responding with harmful content.

There are several different libraries that enable us to be able to build such a system, and here we will introduce the following:

- LangChain
- Haystack
- LlamaIndex
- Semantic Kernel
- AutoGen

Let's look at each of them.

LangChain

LangChain is a framework for developing applications with LLMs at their core. The focus of this framework is the development and deployment of these applications into production. The LangChain ecosystem consists of three core components:

- **LangChain:** Different modules allow the incorporation of LLMs with added memory, prompts, and other tools
- **LangSmith:** This component is used to inspect, monitor, and evaluate your application
- **LangServe:** This allows you to turn your system into an API

LangChain can be used in both Python and JavaScript (and some modules are also available in Rust). To date, it is one of the most widely used libraries in the community, and in fact, there are several components that have been developed by the open source community.

LangChain is compatible with either models that are closed source (such as OpenAI or Anthropic) or models that are available on Hugging Face. LangChain is development-oriented (one of the advantages of LangChain is that it allows both parallel execution and asynchronous support) and one of the best libraries for building an application that needs to go into production. LangChain provides convenient wrappers for LLMs and allows them to be connected to additional tools. One of the most interesting aspects is that it allows you to build so-called chains (LLMs and add-ons) that can then be tracked and deployed in production. LangChain also provides several functions to transform different data (CSV, PDF, text, images, and so on). In addition, the library provides a number of prompt templates to better use the various LLMs.

LangChain creates modular abstractions, thus allowing models to be connected to tools efficiently. By building with chains, you can create efficient (but still versatile and customized) pipelines that can then be easily deployed. In addition, through LangSmith, you can monitor the system to avoid problems.

LangChain has several advantages:

- **Comprehensive library:** It presents a broad library of features with ready-made templates for many applications. In addition, the design is modular, so you can easily swap components.
- **Extensive integrations:** LangChain offers the ability to connect to a large number of external libraries in an easy way: LLM providers, vector databases, cloud service, and so on.
- **Precise and clear workflows:** LangChain makes it possible to clearly define inputs and outputs and also allows intermediate products in the chain to be monitored and extensive prompt engineering to be conducted.
- **Active developing community:** There is a large user base, with different solutions that have been developed by the community, and there are many tutorials written on various sites and forums.
- **Flexible framework focused on an end-to-end cycle:** LangChain provides elements for the entire cycle of an application (integration, development, deployment, and observability).

At the same time, it also has a couple of disadvantages:

- **Steeper learning curve:** Users may require more time to adapt to the library syntax and achieve the full capability of the library. Abstraction capability comes at a cost; all functions are defined as a class. For example, a simple prompt must be abstracted into a **prompt template**.
- **Documentation:** Many users have complained that the documentation is out of date or not easily understood, generalized but not specialized. Versatility is also a disadvantage because, for several specific applications, there are systems that have more functionality (for example, for RAG applications).

LangChain is the most widely used library, especially for building complex agents. However, it also has the steepest learning curve. For this reason, many project users often prefer a simpler library.

Haystack

Haystack is an open source framework for building production-ready LLM applications. Like LangChain, it is compatible with the major LLM sources and deployment platforms. Haystack also allows you to connect tools to LLMs and has a whole set of tools designed to put the system into production (including evaluation, monitoring, and data ingestion). It is designed to be able to easily create LLMs with associated external storage, chatbots, and agents, but also multimodal systems. One of the advantages of Haystack is that it has several pre-built features that can then be inserted into one's own pipeline with ease.

Haystack is built on the idea that everything can be composable with ease, the main elements being the following:

- **Components:** These are building blocks dedicated to document retrieval, text generation, or creating embeddings. These components can be viewed as nodes, and the library presents many that have already been built and are ready to use. The user still has the option of creating their own nodes.
- **Pipelines:** These are a convenient abstraction for understanding how data flows in the application. A pipeline consists of several components that are connected. Haystack facilitates the system because it allows for versatile pipeline control (you can join pipelines, create loops, and so on). In Haystack, you can see them as graphs where the components are nodes that can be interconnected in sophisticated ways.

Haystack has several advantages:

- **Specialized components:** Haystack provides a number of excellent components for data processing, embedding, ranking, and writing. In addition, the library specializes in searching and **question and answer (Q&A)** systems. For this user case, it provides components that are optimized.
- **Extensive documentation and community:** Haystack is adopted by a large community and there are now many community-developed components. It also presents quality documentation and there are many tutorials.
- **Gentler learning curve:** Haystack is considered an easy-to-learn framework. It is versatile, and it is easy to adapt it to different cases.

However, it also has several disadvantages:

- **Smaller user base:** The community is active but smaller than other frameworks such as LangChain or LlamaIndex.
- **Less integration:** There are fewer dedicated integrations than other frameworks. Despite this, the system is flexible and many custom tools

exist.

- **Narrower scope:** Haystack is more focused on retrieval and document-understanding tasks, so it has fewer tools and parsers for other NLP applications. This is a limitation when you have to develop applications that include dialogues, chatbots, or other tools.
- **Scalability:** Many users complain of problems when they have to scale the system or have to handle large datasets.

Haystack is an easy library and can be a great choice for RAG-based applications, but less so for more sophisticated applications that involve agents.

LlamaIndex

LlamaIndex is another framework focused on building a system around an LLM. LlamaIndex began as a data framework that focuses on building RAG. For this reason, the system has several data connectors to both integrate external sources and ingest different types of data. One of the interesting points of LlamaIndex is that it allows knowledge graphs to be easily integrated as well. It can also be integrated with different types of models, but it also has integration for other frameworks (Docker, OpenAI, LangChain, Flask, and so on).

LlamaIndex can be used to nimbly build chatbots and connect them with external storage. It also allows you to build autonomous agents that can search the internet or conduct actions. There are several tools and features already constituted and others that have been developed by the community around it.

LlamaIndex also has several advantages:

- **Handling different data sources:** LlamaIndex can handle over 160 data sources, making it efficient for many types of data commonly found in the enterprise. It is ideal for when you have complex and diverse datasets. It also supports multimodal integration.
- **Indexing and efficient retrieval:** This is the strength of LlamaIndex; it was designed with accurate and fast retrieval of information in mind. The library offers several tools and features for RAG and other retrieval paradigms.
- **Customization:** Especially for retrieval, LlamaIndex offers a high possibility of customization.

There are also some disadvantages:

- **Complexity:** LlamaIndex has a steep learning curve compared to other frameworks. In order to use it best, it assumes that you have a clear idea of information retrieval.
- **Limited functionality:** LlamaIndex has a focus on retrieval tasks but has limited functionality regarding other NLP tasks. This results in a lack of versatility.

LlamaIndex is the first choice for RAG-based applications and a good solution for agents.

Semantic Kernel

Semantic Kernel is an open source framework developed by Microsoft to build agents. This library can also connect with OpenAI, Hugging Face, and other frameworks. Semantic Kernel was originally written in C#, but today there is also a version in Python. The idea behind this library is to provide the ability to create functions that are the result of combining various functions (also known as **function composition**). In other words, Semantic Kernel is structured on the idea that various components can be tied together to build versatile pipelines.

In Semantic Kernel, the core is an LLM, but we can add code that we have developed as plugins so that the LLM can then execute it. In addition, it allows an LLM to have memory that can be either in the form of files or vector databases. An interesting element is that one can create native functions that are dedicated to performing a task. It also implements a planner that takes your task as input and returns a set of actions, functions, or plugins to succeed in solving the task.

Semantic Kernel is versatile, supports several libraries, and has light-weight support for C# and .NET frameworks. It is inspired by the Copilot framework, which is stable and a good choice for enterprises.

AutoGen

AutoGen is a Python-based LLM framework developed by Microsoft and other entities (University of Washington, Penn State, and Xidian). The idea behind it is to build applications that configure multiple agents that communicate with each other to complete a task. Agents and applications work through conversational programming, and a chain is built from the agents' interactions. There are three types of AutoGen agents:

UserProxyAgent (which collects information from users and passes the information to other agents), **AssistantAgent** (which receives data from another **AssistantAgent** instance and **UserProxyAgent**, processes it,

and completes a task), and **GroupChatManager** (which controls and directs communication between agents). AutoGen supports several complex conversation patterns that allow complex workflows to take place without human intervention. Systems involving several agents communicating with each other in complex ways can be configured.

This library has the following advantages:

- **Simplicity:** Abstraction makes it possible to intuit how agents converse and arrive at accomplishing a task. In addition, this makes it easier to explain the system to non-technical staff and other stakeholders.
- **Customization:** The process is intuitive and allows easy customization with little code.

There are a couple of drawbacks, however:

- **Harder to debug:** Agents are interdependent, making debugging difficult
- **Less support:** It is less adopted by the community, so there are fewer users to turn to for help when you need it

AutoGen is an interesting and promising library, but at this stage, it can be hard to start a project with.

Choosing an LLM agent framework

In general, the different frameworks offer similar components and are inspired by the same philosophy (extending the capabilities of an LLM). In addition, almost all libraries today are mature and have purpose-built components. For those functions that are not natively present within the library, there are many resources today that have been built by the community. The main component is the LLM, and the capabilities of the LLM are those that most condition the result of the application.

The first factor that may affect the choice is the programming language of the library. Often, these libraries must be integrated into existing systems that cannot be modified. Almost all libraries are written in Python but also have modules that can be written in other languages and facilitate their integration. In some cases, although support is not native, it has been developed by the open source community. For example, LangChain has support for Rust and there are unofficial implementations in other languages (C#, R, and so on). The tasks that the system has to accomplish are another determining factor. The increased complexity of the system requires that the framework be both robust and flexible at the same time. Some systems are designed with a greater focus on information retrieval

(LlamaIndex) and thus are better choices if our system is to focus on chat and retrieval. In other contexts, we are more interested in system scalability and performance, so we might be interested in all the monitoring and evaluation ecosystems that LangChain and Haystack provide.

In addition, it is desirable that there be an active community of developers and that the library itself be actively maintained. When one adopts a framework, it is important that there are resources and a community to ask to avoid getting stuck or not finding a solution to a bug. A community-adopted library will have a large supply of tutorials and examples to help you learn the system.

Another factor is the level of customization. Although all libraries offer predefined features, these features do not cover all user cases, and in-house solutions will need to be developed. An ideal library should have this versatility and the ability to modify and integrate components. It may happen that we want to change our LLM or one of the components because we need a different performance. Similarly, the default parameters may not be optimal for our application, and it is better to have a library where it is not complex to adapt the parameters.

The best library is determined mainly by the use case. Each of them has strengths and weaknesses. For example, if the application is focused on retrieval, LlamaIndex may be the winning choice, or Haystack if the core of the application is Q&A. LangChain is a more natural choice for a broad scope, but if the system is to be integrated into .NET, you might choose Semantic Kernel. The chosen framework should be evaluated taking into consideration additional constraints and what the main focus of the application is.

We have now seen the most important libraries for AI agents. In the next section, we will start to test them in action.

Creating an agent to search the web

We typically associate internet searches with search engines like Google. The search for a query is conducted using an algorithm, but it is not an AI algorithm. PageRank is, in fact, a graph search algorithm that does not involve learning. A search algorithm presupposes two main steps:

- **Matching:** Finding documents that are relevant to a given user query

- **Ranking:** Ordering these documents from most relevant to least relevant

These two steps do not necessarily require AI. However, searching with the use of AI can bring a better service and solve some of the problems that plague search algorithms. A user today expects an AI algorithm to be able to distinguish the entity and terminology and to contextualize and localize it. For example, a user searching for “the best pizzeria” expects the search engine to return the best restaurants nearby. The search process of the future will also integrate other elements, such as a conversation about the results, complex responses (summarization and action), and multimodality.

There is another aspect. In today’s systems, search results should in many cases be user-related. Searches can be ambiguous and some results will be more relevant by considering the user’s history. In addition, a user may want a search conditional on other parameters and want to express it in natural language (for example, the difference between “best pizzeria in Paris” and expressing “best pizzeria AND Paris”). Once the results are found, the user may have other questions that require reasoning (“Which of these pizzerias is suitable for a dinner with children?”).

An AI-enhanced search can meet these needs because it has an LLM at its core. An LLM can understand the difference between the various keywords in the query, as well as understand which domain the user is looking for (for example, “Transformer” can be an AI model or a toy). In addition, by accessing the history of previous interactions, the LLM is aware of the user’s preferences (so you do not have to state your preferences each time). This allows for a more relevant ranking of the results since they are in order not only for the query but also for a user’s preferences. The model can also conduct reasoning and give more importance to results that are more implicitly relevant (for example, when searching for a family restaurant, a multimodal model will give more preference to a restaurant with a picture of a playground even if it is not explicitly stated in the description that it is for families). The user can also ask questions about the results and the model using agents can perform operations.

The traditional search returns links in order of importance for a query. Today, however, information can be extracted from sites and analyzed by an LLM. For a given query, the model can now either extract the most relevant passage in the site (**extractive question-answering**) or propose a summary of the first results that directly answer the query (**abstractive question-answering**). In this way, the user does not even have to click the links but has the answer directly. This system can then be integrated

with other tools, such as external memory (such as RAG and knowledge graphs, which we will see in the next three chapters).

In addition, an LLM has generative capabilities, so these can be used in combination with the query (“search for sources on the French Revolution” or “search for code in Python for convolution and translate it into R”). One of the problems with this type of generative search is the risk of hallucination, so the system must preserve the sources it has used so that backtracking can be conducted.

In the simplest form of searching with an LLM plus agents, we have an LLM connected with an interface that allows it to receive a query and a tool that allows it to search the internet. In this basic case, the LLM must analyze the query, plan to use the tool to search the internet, analyze the results found, and answer the query. In more sophisticated forms, the LLM may have more tools at its disposal. For instance, it can execute code, a calculator, and external memory to save data or call up models that perform NLP tasks (entity identification in text, extracting passages, and so on).

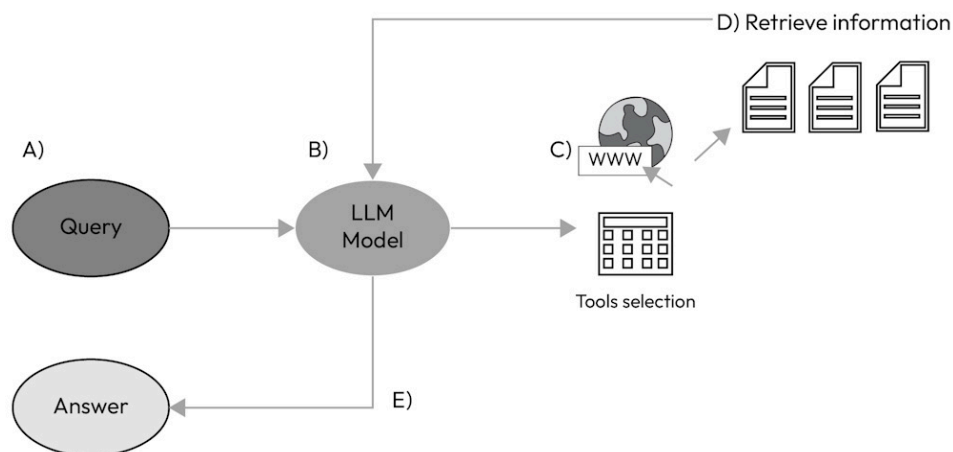


Figure 4.11 – Representation of an LLM agent’s system for internet research

Let’s break down *Figure 4.11*:

1. The user formulates a query.
2. The model analyzes the query and plans actions to solve the task.
3. The appropriate tool (in this case, internet search) is selected.
4. Documents are identified during the internet search. The information is sent back to the model, which analyzes it and decides whether further action is required or whether the task is accomplished.
5. The model generates the answer, which is sent back to the user.

LangChain allows the LLM to be able to connect to the web through the use of tools. In general, the most widely used approach is known as

Reasoning and Acting (ReAct) prompting. During the first stage (reasoning), the model considers the best strategy to be able to arrive at the answer, and in the second stage (acting), it executes the plan. In this approach, the model also tracks the reasoning steps and can guide it to the solution. This approach also allows flexibility because it enables the model to mold the prompt to its needs.

By itself, LangChain offers a number of tools that are designed to extend the model and find the information it needs. For example, it offers several tools for searching the internet. The DuckDuckGo tool allows people to use the DuckDuckGo search engine. This search engine is free and does not track user data (it also filters out pages that are full of advertising or have articles written only to rank highly in the Google search engine).

In order to use it, we need to install a specific Python library and then import the tool into LangChain:

```
!pip install duckduckgo-search
from langchain.tools import DuckDuckGoSearchRun
ddg_search = DuckDuckGoSearchRun()
ddg_search.run('Who is the current president of Italy?')
```

NOTE

You can directly use the tool, and it will provide you with the search results.

One of the strengths of LangChain is the ability to build a list of tools that can then be used by the LLM. To do this, we have to give a name, explain what the function to be performed is, and provide a description. This way, the LLM is informed about what tool it can use when it has to do an internet search:

```
from langchain.agents import Tool
tools = [
    Tool(
        name="DuckDuckGo Search",
        func=ddg_search.run,
        description="A web search tool to extract information from Inter
    )
]
```

It is possible to use Google Serper, a low-cost API (albeit with some limitations) that enables you to use the Google search engine (you have to regis-

ter at <https://serper.dev/> to get an API key); however, there is a fee for the service. In fact, the Google API is much more expensive and the LLM cannot access directly the search engine (Serper allows us to use the Google search engine through their API, and they provide some free credits):

```
#!/pip install google-serp-api
import os
SERPER_API_KEY = 'your_key'
os.environ["SERPER_API_KEY"] = SERPER_API_KEY
from langchain.utilities import GoogleSerperAPIWrapper
google_search = GoogleSerperAPIWrapper()
tools.append(
    Tool(
        name="Google Web Search",
        func=google_search.run,
        description="Google search tool to extract information from Inte
    )
)
```

Additionally, we can add Wikipedia as a reliable source of information:

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
tools.append(
    Tool(
        name="Wikipedia Web Search",
        func=wikipedia.run,
        description="Useful tool to search Wikipedia.",
    )
)
```

Once we have the various tools, we can then initialize an agent and conduct a query:

```
from langchain.agents import initialize_agent
agent = initialize_agent(
    tools, llm, agent="zero-shot-react-description",
    verbose=True
)
agent.run('Query')
```

The model will choose which results will be interesting to our query. The model can access different information but has nowhere to save it. Using

an LLM for internet research can be useful for different fields (from medicine to finance). For example, models have been used to access genetic sequences and conduct comparisons, or to search for information about drugs, chemical structures, and more. Similarly, a model can search for the latest financial and economic news. Another tool can be the **OpenStreetMap (OSM)** search.

In the next chapter, we will discuss how the model can save and access this memory. For example, we may want our model to be able to access conversation history or extend its knowledge. This can be useful in both business applications, but also in fields such as finance and healthcare.

Summary

In this chapter, we introduced how an LLM can be the brain of a sophisticated and complex system. We can use the conversational and reasoning abilities of the LLM to solve a task. As we said, this brain can be extended by providing perceptual systems (senses) and tools (hands). In fact, we can allow the model to search the internet by connecting with APIs, but also to ingest information from other modalities (audio, images, or video). Similarly, the model uses this received information to solve user tasks. If we can imagine agents performing and automating routine tasks for users today, it is not difficult to imagine a world in which agents interact with humans and other agents in increasingly sophisticated and complex ways.

In the next chapter, we will see how a model can have a memory, as well as how to store information and be able to find it again to be more efficient.

Further reading

- Silver, *Mastering the game of Go without human knowledge*, 2017: <https://www.nature.com/articles/nature24270>
- LangChain: <https://python.langchain.com/v0.2/docs/introduction/>
- Haystack: <https://haystack.deepset.ai/overview/intro>
- Semantic Kernel: <https://learn.microsoft.com/en-us/semantic-kernel/overview/>
- AutoGen: <https://microsoft.github.io/autogen/>
- LlamaIndex: <https://www.llamaindex.ai/>
- LangChain tools: <https://python.langchain.com/v0.2/docs/integrations/tools/>
- DuckDuckGo: <https://duckduckgo.com/>

- Guardian, *'Godfather of AI' shortens odds of the technology wiping out humanity over next 30 years*, 2024:

<https://www.theguardian.com/technology/2024/dec/27/godfather-of-ai-raises-odds-of-the-technology-wiping-out-humanity-over-next-30-years>

Table of contents collapsed